

Documentación Estructurada de Arquitectura Lógica y Diagramas de Flujo de Automatización

Benemérita Universidad Autónoma de Puebla

Autor: GALEANA LETRAS EMMANUEL

Profesor: ZENTENO - VAZQUEZ ANA CLAUDIA

3 de junio de 2026

Resumen

Este documento técnico constituye un compendio aislado y formal de la arquitectura de infraestructura y la lógica secuencial implementada en el proyecto de auditoría y endurecimiento (Hardening). A través de modelos vectoriales con enrutamiento de precisión, diagramas de flujo y un análisis forense de los comandos utilizados, se detalla el ecosistema del laboratorio, garantizando que el diseño y la remediación sean comprensibles tanto para perfiles de ingeniería como para directivos sin formación técnica.

Índice

1. Glosario Básico de Conceptos	2
2. Modelado de la Arquitectura e Interacción de Componentes	2
3. Ciclo de Datos y Telemetría de Vulnerabilidades	3
4. Análisis Operacional y Comparativo: Lynis vs. Greenbone	4
4.1. Tabla Comparativa de Herramientas	4
4.2. Diagrama de Flujo: Evaluación Paralela y Remediación	5
5. Modelado y Análisis Forense: Script Lineal (hardening.sh)	6
5.1. Análisis Detallado de Comandos (hardening.sh)	6
6. Modelado y Análisis Forense: Script Modular (hardening_pro.sh)	8
6.1. Análisis Detallado de Comandos (hardening_pro.sh)	8

1. Glosario Básico de Conceptos

Para facilitar la comprensión de este documento a cualquier lector, se definen los siguientes términos clave:

- **Hardening (Endurecimiento):** Es el proceso informático de asegurar un sistema reduciendo sus puntos débiles. Equivale a instalarle rejas, cerraduras de alta seguridad y detectores de movimiento a una casa.
- **Vulnerabilidad (CVE):** Un fallo de seguridad que un atacante puede aprovechar para infiltrarse. Un CVE es simplemente el "nombre de catálogo" mundial que se le da a ese fallo para su correcta identificación.
- **Script de Automatización:** Un archivo que contiene una lista de instrucciones que la computadora ejecuta automáticamente, de arriba hacia abajo, ahorrando trabajo manual y evitando errores humanos.
- **Host (Anfitrión) y M.V. (Máquina Virtual):** El Host es la computadora física real que tocas con las manos. La Máquina Virtual es una "computadora emulada" por software que vive dentro del Host, como si fuera un holograma o una simulación.

2. Modelado de la Arquitectura e Interacción de Componentes

Nota para el Lector General: ¿Qué es la Arquitectura de Virtualización?

Imagina que la computadora física (Windows 11) es un gran **terreno vacío**. En lugar de construir un solo edificio gigante que mezcle todo, usamos herramientas (VirtualBox y Docker) para dividir el terreno y construir dos **casas completamente aisladas** e independientes. En una casa vive el servidor que queremos proteger (Debian), y en la otra casa vive el equipo de seguridad que lo va a auditar (Greenbone). Si algo explota en una, la otra no sufre ningún daño.

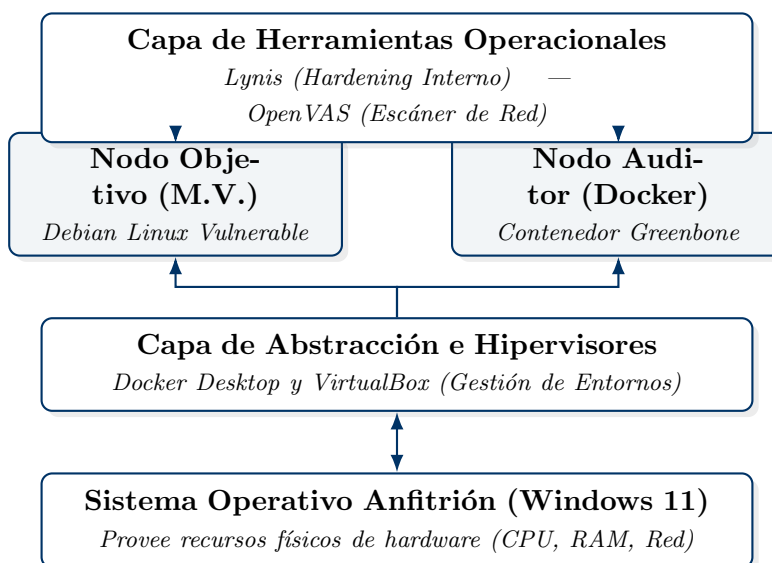


Figura 1: Abstracción jerárquica de componentes de hardware y entornos virtuales.

3. Ciclo de Datos y Telemetría de Vulnerabilidades

Nota para el Lector General: ¿Cómo fluyen los datos de seguridad?

Imagina que la **Base de Datos** es el manual mundial donde los ladrones publican sus nuevos métodos para abrir puertas. Nuestro motor (Greenbone) descarga ese manual, y luego va al servidor a intentar usar esos trucos desde la calle. Por otro lado, Lynis no lee manuales externos, sino que entra a la casa y revisa minuciosamente si dejamos la estufa encendida. Al final, ambos envían sus reportes a la mesa de control.

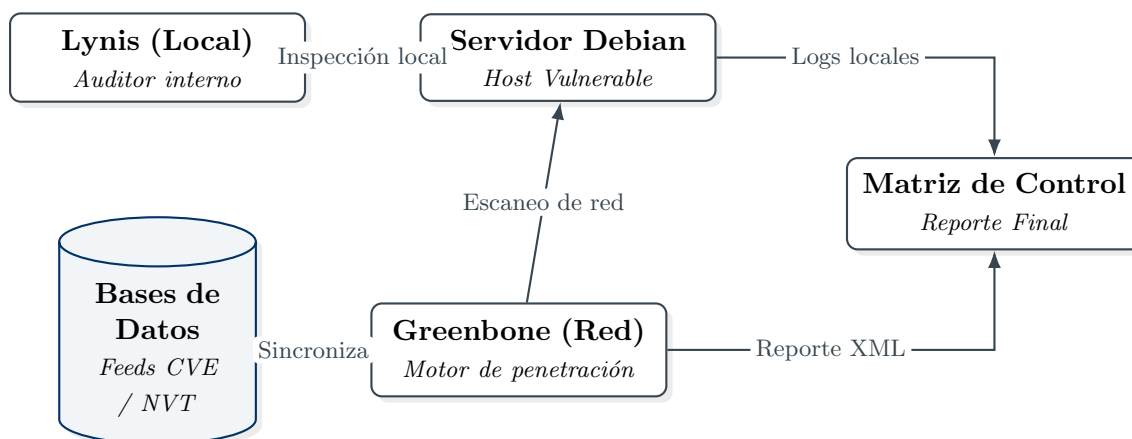


Figura 2: Arquitectura del flujo de inyección de pruebas y recopilación de métricas.

4. Análisis Operacional y Comparativo: Lynis vs. Greenbone

Es vital comprender las diferencias metodológicas de las dos herramientas utilizadas en el laboratorio para garantizar la detección total de brechas.

Nota para el Lector General: El Guardia de Seguridad vs. El Inspector Interno

Greenbone (Escáner de Red): Actúa como un **Guardia de Seguridad** patrullando la calle. Va empujando las puertas y ventanas de la casa desde el exterior para ver si alguna está abierta o es fácil de romper. Sin embargo, no puede ver qué pasa adentro.

Lynis (Auditor Local): Actúa como un **Inspector de Protección Civil**. Él tiene las llaves de la casa, entra de manera autorizada y revisa si los extintores sirven, si los cables eléctricos están pelados y si la caja fuerte interna está bien cerrada.

4.1. Tabla Comparativa de Herramientas

Cuadro 1: Matriz comparativa de capacidades de auditoría de seguridad.

Característica	Greenbone (OpenVAS)	Lynis Core
Enfoque de Análisis	Externo / Escáner de Red (Caja Negra)	Interno / Auditoría de Host (Caja Blanca)
Motor de Reglas	Base de datos externa (CVE / NVT)	Reglas estáticas (CIS Benchmarks)
Modo de Ejecución	Remoto (A través de la red TCP/UDP)	Local (Ejecución directa como Root)
Objetivo Principal	Detectar software desactualizado y vulnerabilidades de red.	Detectar malas configuraciones locales y permisos débiles.
Impacto en Sistema	Inyecta tráfico de red simulando un ataque real.	Pasivo. Lee archivos internos y verifica los logs del sistema.

4.2. Diagrama de Flujo: Evaluación Paralela y Remediación

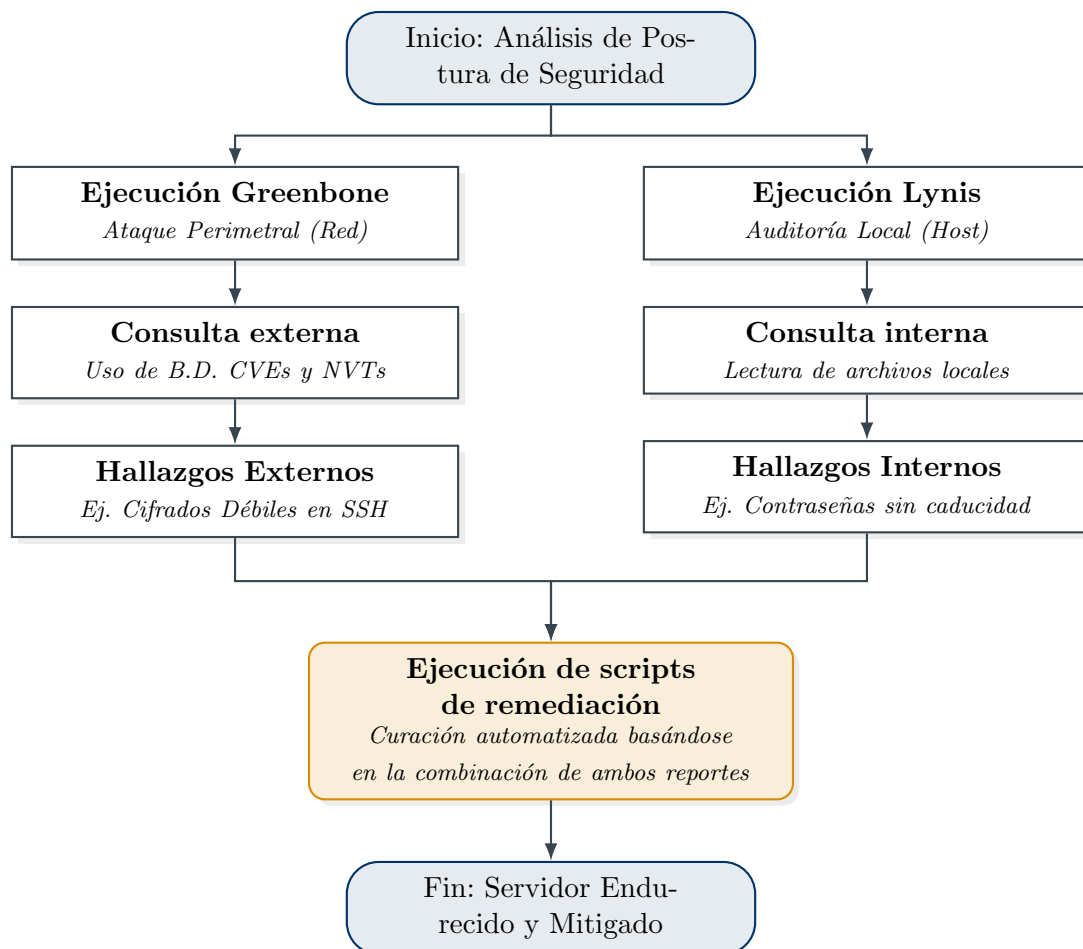


Figura 3: Convergencia del flujo de auditoría hacia la automatización.

5. Modelado y Análisis Forense: Script Lineal (hardening.sh)

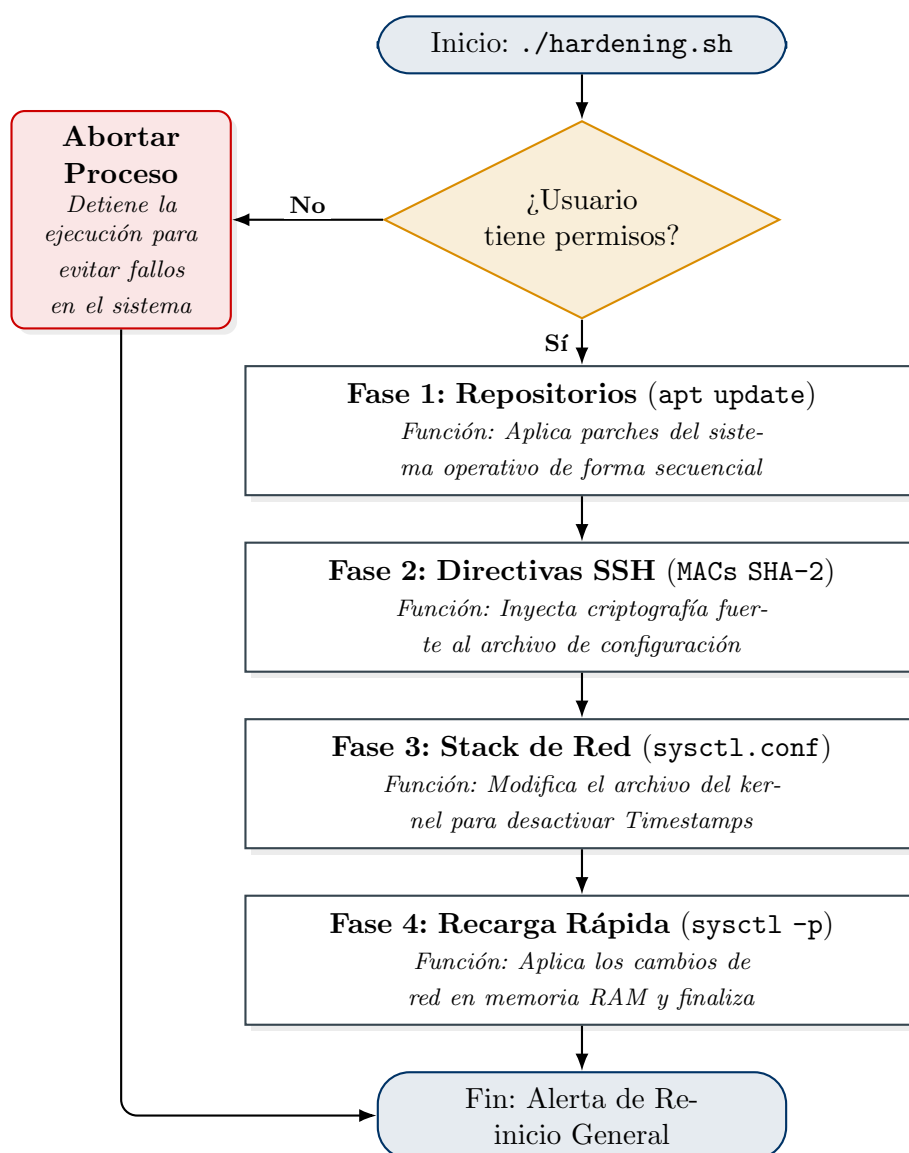


Figura 4: El script avanza de arriba hacia abajo sin segmentación de fallos.

5.1. Análisis Detallado de Comandos (hardening.sh)

Nota para el Lector General: ¿Qué significa un Código Lineal?

El Script Lineal ('hardening.sh'): Es como una **lista de compras rígida y automatizada**. La computadora entra al supermercado y empieza a echar cosas al carrito una tras otra a toda velocidad sin revisar si los pasillos existen. Si no encuentra el pasillo de las manzanas, el script no sabe qué hacer, genera un error, pero sigue corriendo a ciegas intentando comprar el resto de las cosas. Esto puede causar daños estructurales graves al sistema si un comando falla.

A continuación, se presenta la autopsia técnica de cada instrucción inyectada en esta versión inicial:

Comando a Nivel de Sistema: `apt update && apt upgrade -y`

Acción Técnica: Sincroniza los repositorios de Linux y actualiza todos los paquetes de software instalados a su versión más reciente, forzando la confirmación afirmativa con el sufijo `-y`.

Problema que Soluciona: Elimina vulnerabilidades críticas (CVEs) documentadas que los atacantes utilizan para penetrar sistemas desactualizados.

Analogía Explicativa: Es como llevar tu coche a la agencia automotriz para que le apliquen un llamado a revisión (recall) porque los fabricantes descubrieron que los frenos que te instalaron de fábrica podían fallar.

Comando a Nivel de Sistema: `echo "MACs hmac-sha2-512..." >> /etc/ssh/sshd_config`

Acción Técnica: Utiliza la orden `echo` para concatenar (`>>`) forzosamente una línea de texto al final del archivo de configuración del servicio remoto SSH, obligando al servidor a rechazar criptografía antigua.

Problema que Soluciona: Impide que un hacker realice un ataque de intermediario (*Man-in-the-Middle*) logrando descifrar las credenciales porque la matemática del algoritmo original (Weak MAC) era muy débil.

Analogía Explicativa: Es como arrancar el candado de combinación barato que tenías en la puerta principal de tu empresa y reemplazarlo a la fuerza por una bóveda con cerradura biométrica y reconocimiento facial.

Comando a Nivel de Sistema: `net.ipv4.tcp_timestamps = 0 → sysctl -p`

Acción Técnica: Modifica los parámetros profundos del núcleo del sistema operativo (*Kernel*). El comando `sysctl -p` obliga al servidor a inyectar este cambio inmediatamente en la memoria RAM en caliente, sin necesidad de reiniciar el servidor físicamente.

Problema que Soluciona: Detiene la vulnerabilidad *TCP Timestamps Disclosure*. Anteriormente, el servidor revelaba su "Uptime" (tiempo exacto desde que se encendió). Los atacantes usan este dato para calcular secuencias lógicas y evadir barreras de defensa.

Analogía Explicativa: Es como quitarle la etiqueta con tu nombre, teléfono y dirección a tu maleta cuando viajas en avión. Estás evitando regalarle información confidencial gratuita a un extraño que esté observando.

6. Modelado y Análisis Forense: Script Modular (hardening_pro.sh)

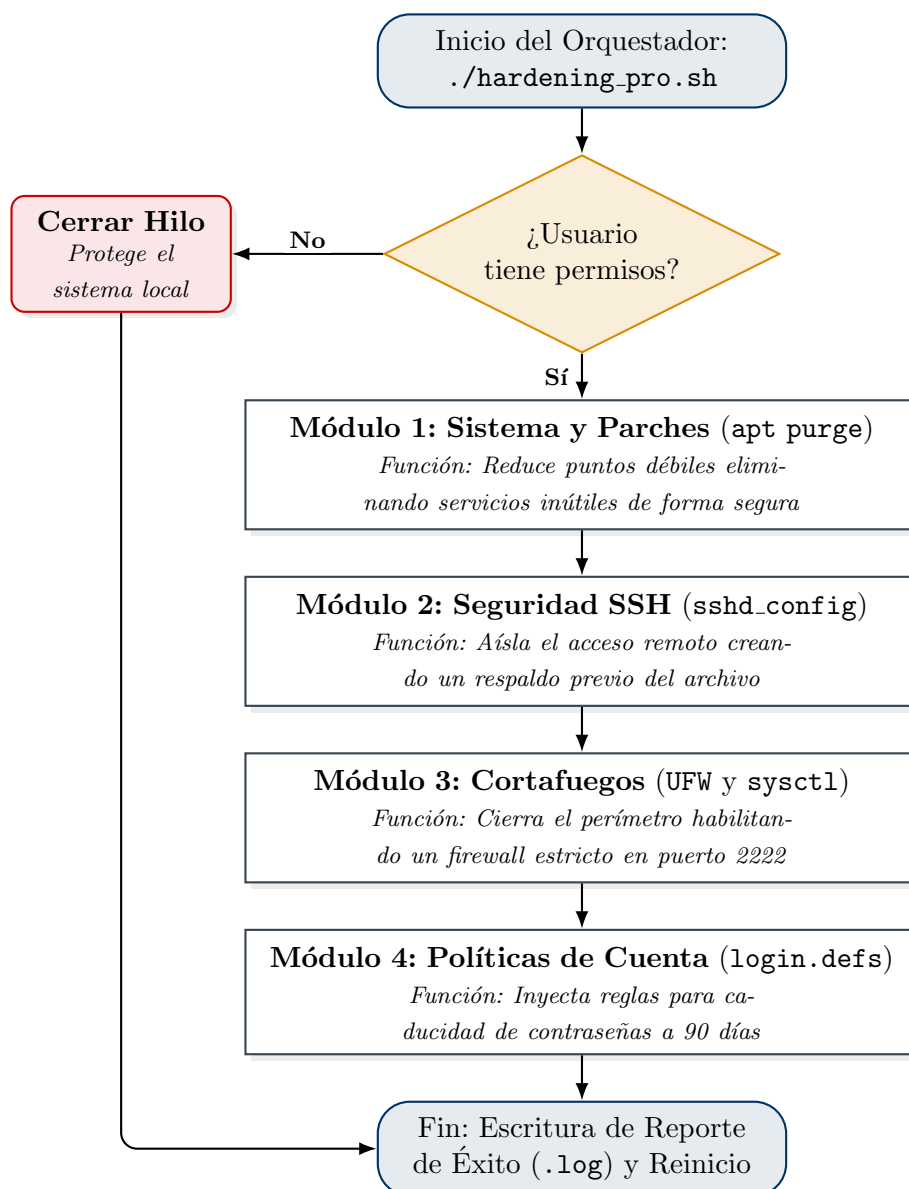


Figura 5: El framework aísla procesos y registra su éxito de manera modular.

6.1. Análisis Detallado de Comandos (hardening_pro.sh)

La evolución arquitectónica a este código incorpora prevención de fallos, modularidad de funciones y registro de trazabilidad.

Nota para el Lector General: ¿Qué es la Programación Modular Orquestada?

El Script Modular ('hardening_pro.sh'): Es el equivalente a contratar a un **Capataz de Obra Profesional** (Orquestador Principal). Él no intenta hacer todo a la vez; primero llama al experto en plomería (Módulo 1) y verifica que su labor terminó sin fugas. Luego llama al electricista (Módulo 2). Si una tubería falla, el capataz cancela esa sección específica, lo documenta detalladamente en una libreta de auditoría (Archivos Log) y protege el resto del proyecto en lugar de tirar la casa entera.

Comando a Nivel de Sistema: `apt purge rpcbind nis -y && apt autoremove`

Acción Técnica: Invoca el gestor de paquetes para desinstalar de raíz (**purge**) servicios heredados que se instalan por defecto pero no se utilizan (**rpcbind** y **nis**), destruyendo también su configuración. **autoremove** barre dependencias huérfanas o "archivos basura".

Problema que Soluciona: Elimina "Puertos Fantasma" en el servidor que los atacantes usan para escanear y mapear la estructura interna de la red de la empresa.

Analogía Explicativa: Es como tapar con ladrillos y cemento una vieja puerta trasera de tu casa que daba a un callejón oscuro. Como nunca la usabas y la madera estaba podrida, lo más seguro es tapiarla por completo para que nadie intente patearla y entrar.

Comando a Nivel de Sistema: `sed -i 's/^PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config`

Acción Técnica: Utiliza la herramienta **sed** (un editor de texto que trabaja de forma invisible) para buscar la línea de código exacta que controla el permiso del usuario Root, y la sobrescribe de forma milimétrica reemplazándola por un rotundo "no".

Problema que Soluciona: Evita el compromiso total del sistema impidiendo que los hackers usen diccionarios automatizados para adivinar la contraseña del administrador supremo (*Fuerza Bruta*).

Analogía Explicativa: Es como dar la orden estricta a los guardias de que el Dueño Principal de la empresa nunca puede entrar al edificio directamente desde la puerta de la calle. Ahora, por seguridad, debe entrar vestido de civil, pasar por recepción, identificarse y pedir las llaves de la bóveda desde adentro.

Comando a Nivel de Sistema: `ufw default deny incoming → ufw allow 2222/tcp → ufw enable`

Acción Técnica: Configura el cortafuegos del núcleo de Linux (UFW). Primero, declara una política radical bloqueando (**deny**) absolutamente todo el tráfico de internet. Segundo, abre una excepción controlada permitiendo únicamente el tráfico de gestión hacia el puerto 2222. Finalmente, enciende el motor (**enable**).

Problema que Soluciona: Vuelve "invisible" al servidor. Cualquier servicio inseguro o virus que intente comunicarse con internet rebotará contra el muro del Firewall sin obtener respuesta.

Analogía Explicativa: Es como construir un muro ciego, liso e indestructible de 10 metros de altura alrededor de todo tu corporativo, dejando únicamente una diminuta puerta blindada custodiada por un guardia que exige una invitación sellada para poder cruzar.

Comando a Nivel de Sistema: `sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS 90/' /etc/login.defs`

Acción Técnica: Modifica el archivo maestro de identidades de Linux (**login.defs**) para alterar el ciclo de vida criptográfico de las cuentas. Impone a nivel de sistema operativo que la duración máxima absoluta de una credencial antes de exigir un cambio obligatorio sea de 90 días exactos.

Problema que Soluciona: Cubre los hallazgos críticos detectados por la herramienta **Lynis**, garantizando que si un empleado es víctima de robo de contraseñas (Phishing), el atacante pierda el acceso automáticamente cuando caduque el umbral de tiempo.

Analogía Explicativa: Imagina la tarjeta magnética que te entregan en la recepción de un hotel. Esa tarjeta se desactiva sola en un par de días para asegurar que, si la pierdes o te la roban en el metro, el ladrón ya no pueda usarla para abrir la puerta de tu habitación la semana siguiente.